



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Design and Implementation of a 5-Stage Pipelined RISC-V RV32I Processor

The domain of the Project:

RTL Design

Team Mentor (and designation):

Mr.Razi Ahmed

Gmail : md.razi720@gmail.com

Designation : RTL Design Engineer

Team Member:

Suraj-G3-25-Vlsi

Period of the project

May/2026-july/2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Declaration

The project titled “Design and Implementation of a 5-Stage Pipelined RISC-V RV32I Processor” has been mentored by **Razi Ahmed** organised by SURE Trust, from **May 2026 to July 2026** for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge, I have worked on it successfully and enhanced my practical knowledge in the domain of digital design and computer architecture.

Team Member:

Guruprasad

Mentor, SURE Trust

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion



Executive Summary

Modern computing systems rely on pipelined processor architectures to achieve high instruction throughput while maintaining hardware efficiency. This project presents the design, implementation, and functional verification of a classic five-stage pipelined processor core based on the RV32I base integer instruction set of the open-standard RISC-V Instruction Set Architecture (ISA), described entirely in Verilog HDL.

The primary objective was to implement the core R-type, I-type arithmetic, load, store, branch (BEQ, BNE, BLT, BGE), and jump (JAL, JALR) instruction classes, along with the U-type instructions LUI and AUIPC, and to ensure correct operation across back-to-back dependent instructions using data forwarding and hazard-stall logic. The methodology followed a modular RTL design approach, with each pipeline stage and supporting unit implemented as an independent Verilog module and integrated through a top-level datapath. Functional verification was carried out using Synopsys VCS, with a self-checking testbench that compared simulated register-file and data-memory contents against expected architectural results.

During verification, two functional defects were identified and corrected through systematic root-cause analysis: a same-cycle register-file read/write hazard that produced incorrect results for the SUB instruction, resolved by introducing a write-through bypass inside the register file; and a missing datapath path for the AUIPC instruction's PC-relative result, resolved by extending the write-back multiplexer and pipeline registers to carry a dedicated PC + immediate sum. Following these corrections, the processor was validated using a self-checking testbench covering thirty-one architectural checks across all implemented instruction classes and hazard scenarios, with all checks passing. The project provided practical, industry-relevant experience in RTL design, pipeline hazard analysis, pre-silicon debugging methodology, and simulation-based hardware verification.



Introduction

Background and Context

The RISC-V Instruction Set Architecture (ISA) is an open, royalty-free ISA that has become widely adopted in both academia and industry for processor research, education, and commercial silicon development. Pipelining is a fundamental technique used to improve instruction throughput by overlapping the execution of multiple instructions across distinct pipeline stages. A classic five-stage pipeline — Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) — is widely used as the standard teaching and reference architecture for understanding pipeline hazards, forwarding, and control-flow handling in modern processors.

Problem Statement

Designing a pipelined processor introduces hazards not present in single-cycle designs. Data hazards occur when an instruction depends on the result of a preceding instruction that has not yet completed Write-Back; control hazards occur when branch or jump instructions change the program counter after subsequent instructions have already been fetched. The problem addressed by this project is the correct, verifiable implementation of a five-stage RV32I pipeline that detects and resolves these hazards without corrupting architectural state.

Scope and Limitations

The scope of this project covers the RTL design, integration, and functional simulation-based verification of a five-stage pipelined RV32I processor core, including R-type and I-type arithmetic instructions, load and store instructions, conditional branches, unconditional jumps, and the U-type instructions LUI and AUIPC, together with data-hazard forwarding, load-use stalling, and control-hazard flushing. The current implementation targets functional correctness through RTL simulation and has not yet been carried through synthesis, timing closure, or FPGA/ASIC deployment; the instruction and data memories are modelled behaviourally within the testbench rather than as synthesisable memory macros.

Innovation Component

The project's practical contribution lies in its systematic, evidence-based debugging methodology: cycle-by-cycle pipeline-monitor analysis was used to trace an observed SUB-instruction failure to its true root cause — a same-cycle register-file read/write collision — rather than misattributing the defect to the ALU control logic or forwarding unit, both of which were initially, and incorrectly, suspected. This was followed by the development of a self-checking testbench with thirty-one automated architectural assertions, replacing manual log inspection with an automated pass/fail verification methodology suitable for regression testing.



Project Objectives

Objectives and Goals

- Design, implement, and functionally verify a five-stage pipelined RV32I RISC-V processor core in Verilog HDL.
- Implement correct data-hazard resolution through EX/MEM and MEM/WB operand forwarding and load-use hazard stalling.
- Implement correct control-hazard resolution through pipeline flushing on taken branches and unconditional jumps.
- Identify, root-cause, and correct functional defects discovered during simulation-based verification.
- Develop a self-checking testbench capable of automatically validating architectural state against expected results.

Expected Outcomes and Deliverables

- A complete, modular Verilog RTL implementation of a five-stage pipelined RV32I processor core.
- A self-checking Verilog testbench with automated PASS/FAIL reporting across thirty-one architectural checks.
- Documented root-cause analysis and corrective fixes for all functional defects identified during verification.
- Simulation logs and register-file/data-memory dumps demonstrating correct architectural behaviour.



Methodology and Results

4.1 Overall Methodology

The processor was developed individually, following a structured, bottom-up RTL design methodology consisting of the following stages:

1. Study of the RISC-V RV32I base integer instruction set architecture and its instruction encoding formats.
2. Identification of the specific subset of RV32I instructions to be implemented and verified.
3. Design of the single-cycle processor datapath (Program Counter, Register File, ALU, Immediate Generator, Decoder).
4. Design of individual RTL modules in Verilog HDL for each datapath and control component.
5. Integration of the individual modules into a complete single-cycle datapath.
6. Conversion of the single-cycle datapath into a five-stage pipelined datapath by inserting pipeline registers between stages.
7. Identification of data and control hazards introduced by pipelining.
8. Design and implementation of the forwarding unit to resolve data hazards without stalling where possible.
9. Design and implementation of load-use hazard detection and stall logic.
10. Design and implementation of pipeline flush logic for taken branches and jumps.
11. Development of a directed instruction sequence and a self-checking testbench.
12. Simulation of the design using Synopsys VCS.
13. Analysis of pipeline-monitor traces and waveform behaviour.
14. Debugging of functional defects identified during simulation.
15. Final verification of processor functionality against expected architectural results.

4.2 Tools and Software

The following tools and software were used in the design, simulation, and verification of the processor. No tool listed below is claimed beyond what was actually used.

Tool / Software	Purpose
Verilog HDL (IEEE 1364)	Register-transfer-level description of the processor datapath and control logic.
Verilator and GTK wave	Compilation and event-driven simulation of the design and self-checking testbench.
SystemVerilog testbench constructs	Self-checking testbench tasks (check_reg, check_mem) and pipeline monitor for verification.



Operating System

Linux (64-bit).

4.3 System Architecture

The processor implements the classic five-stage RISC pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). The stages are separated by pipeline registers that latch control and data signals on every clock edge, allowing up to five instructions to be in flight simultaneously. The overall data flow through the processor is summarised below.

Figure 1: Overall RISC-V Processor Pipeline Architecture (data-flow order)

Program Counter (PC)



Instruction Fetch (Instruction Memory Interface)



IF/ID Pipeline Register



Instruction Decode + Register File Read + Immediate Generation (Control Unit / Decoder)



ID/EX Pipeline Register



Execute (ALU / Branch Unit / Immediate-PC Adder) ← EX-stage Forwarding Muxes



EX/MEM Pipeline Register



Memory Access (Store Unit / Load Unit ↔ Data Memory Interface)



MEM/WB Pipeline Register



Write Back (Write-Back Multiplexer)



Register File (write port)

A centralised Hazard Detection and Forwarding Unit runs alongside the pipeline shown above. It compares register addresses across the Decode, Execute, Memory, and Write-Back stages to generate: (i) EX-stage forwarding-select signals that route data from the EX/MEM or MEM/WB pipeline registers directly into the ALU inputs, bypassing the Register File; (ii) a load-use stall signal that freezes the Fetch and Decode stages for one cycle when required; and (iii) flush signals that clear the IF/ID and ID/EX pipeline registers when a branch or jump is resolved as taken in the Execute stage.



4.4 Processor Modules

Each functional unit of the processor was implemented as an independent Verilog module. The purpose, key inputs/outputs, and role of each module (using the exact module names from the RTL source) are described below.

pc_unit — Program Counter

Purpose: Maintains the 32-bit program counter and updates it every clock cycle.

Inputs: Stall_F, branch_taken, target_addrs

Outputs: pc_out, pc_plus4_out

Role / Interaction: Selects the next PC as PC+4, a redirected branch/jump target, or the held value during a stall.

if_register — IF/ID Pipeline Register

Purpose: Latches the fetched instruction and PC values from the Fetch stage into the Decode stage.

Inputs: instruction_F, PC_F, PC_plus4_F, Stall_D, Flush_D

Outputs: Instruction_D, PC_D, PCPlus4_D

Role / Interaction: Supports stalling (holds current value) and flushing (inserts a NOP/bubble) under hazard-unit control.

Decoder — Instruction Decoder / Control Unit

Purpose: Decodes the opcode, funct3, and funct7 fields of the instruction to generate all downstream control signals.

Inputs: opcode_in, Func3_in, Func7_in

Outputs: Reg_wr_en_out, imm_type_out, ALU_src_out, ALU_control_out, DM_wr_en, Branch_cond_out, Load_unsigned_out, Load_size_out, Result_src_out

Role / Interaction: Central control unit; every other stage's behaviour for a given instruction is ultimately driven by signals generated here.

register_file — Register File

Purpose: Implements the 32 x 32-bit RV32I general-purpose register set (x0–x31), with x0 hard-wired to zero.

Inputs: wr_en, des_addr_in, des_data_in, src_addr1_in, src_addr2_in

Outputs: src_data1_out, src_data2_out

Role / Interaction: Synchronous write, combinational read, with an internal write-through bypass so that a register being written in Write-Back is correctly forwarded if read in the same cycle by Decode.

imm_gen — Immediate Generator

Purpose: Extracts and sign-extends the immediate field according to instruction format (I, S, B, U, J).

Inputs: instr_in, imm_type



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Outputs: imm_out

Role / Interaction: Produces the correctly sign-extended 32-bit immediate value required by I-type, S-type, B-type, U-type, and J-type instructions.

ID_Register — ID/EX Pipeline Register

Purpose: Latches decoded control signals and operand data from Decode into Execute.

Inputs: Control signals, RD1_D, RD2_D, ImmExt_D, RS1_D, RS2_D, RD_D, Flush_E

Outputs: Corresponding *_E signals

Role / Interaction: Supports flushing to insert a bubble into Execute when a control hazard (taken branch/jump) or load-use stall is detected.

alu — Arithmetic Logic Unit

Purpose: Performs the arithmetic or logic operation selected by the ALU control signal.

Inputs: in1, in2, alu_control

Outputs: alu_result

Role / Interaction: Supports ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, and SLTU, covering all implemented R-type and I-type arithmetic instructions.

imm_adder — Immediate / Branch Address Adder

Purpose: Computes PC + immediate (for branches, JAL, and AUIPC) or rs1 + immediate (for JALR).

Inputs: ladder_src_in, PC_in, src_Data1_in, imm_data_in

Outputs: Added_data_out

Role / Interaction: Shared adder used both for branch/jump target computation and for the AUIPC result.

branch_unit — Branch Condition Unit

Purpose: Evaluates the branch or jump condition for the instruction currently in Execute.

Inputs: src_data1_in, src_data2_in, branch_cond_in

Outputs: branch_taken_out

Role / Interaction: Supports BEQ, BNE, BLT, BGE, JAL, and JALR condition evaluation, driving the pipeline flush and PC-redirect logic.

EX_Register — EX/MEM Pipeline Register

Purpose: Latches the ALU result, store data, and control signals from Execute into Memory.

Inputs: ALUResult_E, WriteData_E, control signals

Outputs: Corresponding *_M signals

Role / Interaction: Passes the address/result computed in Execute forward to the Memory stage for load/store operations or write-back.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

store_unit — Store Unit

Purpose: Generates the data-memory write address, write data, and byte-enable mask for SB/SH/SW instructions.

Inputs: DM_WrEN_in, Func3_in, Added_Data_in, Src_Data2_in

Outputs: DM_Addr_OUT, DM_WrData_OUT, DM_WrMask_out, DM_WrEn_out

Role / Interaction: Produces the correct byte/half-word/word write mask depending on store instruction width.

load_unit — Load Unit

Purpose: Performs sign or zero extension of the value read from data memory, according to load width and signedness.

Inputs: read_data_in, load_size_in, load_unsigned_in

Outputs: loaded_data_out

Role / Interaction: Supports LB, LH, LW, LBU, and LHU by correctly extending the loaded byte/half-word/word.

MEM_WB_Register — MEM/WB Pipeline Register

Purpose: Latches memory-stage results and control signals into the Write-Back stage.

Inputs: ReadData_M, ALUResult_M, control signals

Outputs: Corresponding *_W signals

Role / Interaction: Final pipeline register before the result is selected and written back to the Register File.

wb_mux — Write-Back Multiplexer

Purpose: Selects the value that is ultimately written back to the Register File.

Inputs: result_src_in, alu_result_in, load_data_in, pc_plus4_in, imm_data_in, pc_imm_sum_in

Outputs: wb_data_out

Role / Interaction: Chooses between the ALU result, loaded data, PC+4 (for JAL/JALR), immediate value (for LUI), or PC+immediate (for AUIPC).

Hazard_Unit — Hazard Detection and Forwarding Unit

Purpose: Detects data and control hazards across all pipeline stages and generates the corresponding control signals.

Inputs: Register addresses and RegWrite/ResultSrc signals from ID, EX, MEM, and WB stages; Branch_Taken_E_In

Outputs: Stall_F_Out, Stall_D_Out, Flush_D_Out, Flush_E_Out, ForwardA_E_Out, ForwardB_E_Out

Role / Interaction: The central hazard-resolution unit: generates EX-stage forwarding selects (MEM stage prioritised over WB stage), the load-use stall signal, and branch/jump flush signals.



top_module — Top-Level Datapath

Purpose: Instantiates and interconnects all of the above modules into the complete five-stage pipelined processor.

Inputs: instruction_in, DM_data_in, clk_in, rst_in

Outputs: instr_addr_out, DM_WrEn_out, DM_mask_out, DM_addr_out, DM_data_out

Role / Interaction: Top-level integration point; defines the wiring between every pipeline stage and the Hazard Unit.

4.5 Instruction Formats

RV32I defines six instruction formats, distinguished by how the instruction word's 31 bits (excluding the 7-bit opcode) are interpreted. The Decoder and Immediate Generator modules process each format as follows.

Format	Bit Fields (31...0)	Used By
R-type	funct7 rs2 rs1 funct3 rd opcode	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
I-type	imm[11:0] rs1 funct3 rd opcode	ADDI, ANDI, XORI, ORI, loads, JALR
S-type	imm[11:5] rs2 rs1 funct3 imm[4:0] opcode	SB, SH, SW
B-type	imm[12,10:5] rs2 rs1 funct3 imm[4:1,11] opcode	BEQ, BNE, BLT, BGE
U-type	imm[31:12] rd opcode	LUI, AUIPC
J-type	imm[20,10:1,11,19:12] rd opcode	JAL

The Immediate Generator module (imm_gen) reconstructs the correctly sign-extended 32-bit immediate for each format from the corresponding bit fields, and the Decoder selects the immediate type and all other control signals based on the 7-bit opcode together with funct3 (and, for R-type and shift-type instructions, funct7).

4.6 Instruction Execution Flow

Every instruction, regardless of type, passes through all five pipeline stages IF → ID → EX → MEM → WB, though the work performed in each stage differs by instruction class. Representative examples from the implemented instruction set:

- ADD / SUB (R-type): IF fetches the instruction; ID reads rs1 and rs2 and decodes the ALU operation; EX computes $rs1 + rs2$ or $rs1 - rs2$; MEM performs no memory access; WB writes the ALU result to rd.
- ADDI (I-type): identical to ADD, except the second ALU operand is the sign-extended immediate rather than rs2.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- LW (Load): EX computes the effective address ($rs1 + \text{immediate}$); MEM reads data memory at that address; the Load Unit sign/zero-extends the result; WB writes the extended value to rd.
- SW (Store): EX computes the effective address; MEM writes rs2's value to data memory through the Store Unit; no Write-Back register write occurs.
- BEQ / BNE / BLT / BGE (Branch): EX evaluates the branch condition in the Branch Unit and computes the branch target via the Immediate/Branch Adder; if taken, the Hazard Unit flushes the two younger instructions already fetched and the PC is redirected.

4.7 Pipeline Hazards

Three classes of hazard are possible in a pipelined processor:

- Structural hazards — occur when two instructions require the same hardware resource in the same cycle. The five-stage design used here avoids structural hazards by construction, since each stage has dedicated hardware and separate instruction/data memory interfaces.
- Data hazards — occur when an instruction depends on the result of a preceding instruction that has not yet completed Write-Back. This is the primary hazard class handled by this project, through forwarding and, where forwarding is insufficient, stalling.
- Control hazards — occur when a branch or jump instruction changes the program counter after younger instructions have already been fetched based on the old, sequential PC. This project handles control hazards through pipeline flushing once the branch/jump outcome is resolved in the Execute stage.

Because up to five instructions are simultaneously present in different pipeline stages, a data or control dependency between two nearby instructions can otherwise cause the pipeline to read stale operand values or execute instructions that should not have been fetched.

4.8 Forwarding Unit

Forwarding (also called bypassing) allows a result that has been computed but not yet written back to the Register File to be routed directly to the ALU inputs of a later instruction that needs it, avoiding an unnecessary pipeline stall.

The Hazard Unit compares the Execute-stage source register addresses ($RS1_E$, $RS2_E$) against the destination register addresses of the instructions currently in the Memory stage ($Des_Addr_M_In$) and the Write-Back stage ($Des_Addr_W_In$), provided the corresponding instruction writes to the register file ($Reg_WrEn_M_In$ / $Reg_WrEn_W_In$) and the destination is not x0. Two independent 2-bit selects, $ForwardA_E_Out$ and $ForwardB_E_Out$, are generated:

- 2'b00 — no forwarding; use the value already latched in the ID/EX pipeline register ($RD1_E$ / $RD2_E$).



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- 2'b10 — forward from the EX/MEM pipeline register (ALUResult_M) — Memory-stage forwarding, given priority.
- 2'b01 — forward from the Write-Back stage result (WB_Data) — used only when Memory-stage forwarding does not apply.

Example: in the sequence `ADD x3, x1, x2` followed immediately by `OR x6, x3, x1`, the second instruction depends on `x3`, which is still in the Memory stage when `OR` reaches Execute. The Hazard Unit sets `ForwardA_E = 2'b10`, forwarding the ALU result for `x3` directly into the ALU rather than reading the (not-yet-updated) Register File.

4.9 Stall Unit

Forwarding alone cannot resolve a load-use hazard, because the loaded value is not available until the end of the Memory stage — one cycle later than an ALU result would be. If an instruction immediately following a load depends on the loaded register, the pipeline must stall for one cycle.

The Hazard Unit detects this condition (signal `LW_Stall`) when the instruction in Execute is a load (`Result_Src_E_In = 3'b001`) and its destination register matches either source register of the instruction currently in Decode. When detected, the Hazard Unit asserts `Stall_F_Out` and `Stall_D_Out`, which hold the Fetch and Decode pipeline registers at their current values for one cycle (preventing the PC from advancing and preventing Decode from overwriting its latched instruction), while `Flush_E_Out` simultaneously inserts a bubble (a NOP) into the Execute stage so that no incorrect instruction is executed during the stall cycle.

4.10 Flush Mechanism

Because the processor fetches instructions sequentially before a branch or jump outcome is known, up to two younger instructions may already be present in the IF/ID and ID/EX pipeline registers by the time a taken branch or jump is resolved in the Execute stage.

When the Branch Unit asserts `Branch_Taken_E_In`, the Hazard Unit asserts `Flush_D_Out` (clearing the IF/ID pipeline register, discarding the instruction fetched from the sequential path) and `Flush_E_Out` (clearing the ID/EX pipeline register, discarding the instruction already decoded from the sequential path). Both flushed pipeline registers are reset to a NOP-equivalent state (all control signals de-asserted), ensuring the two incorrectly fetched instructions have no architectural effect, while the Program Counter is redirected to the correct branch/jump target.

4.11 Pipeline Timing

In the absence of hazards, the five-stage pipeline allows one instruction to complete every clock cycle after the pipeline is full, as shown below for five consecutive instructions.

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6
Instr 1	IF	ID	EX	MEM	WB	



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Instr 2		IF	ID	EX	MEM	WB
Instr 3			IF	ID	EX	MEM
Instr 4				IF	ID	EX
Instr 5					IF	ID

When a load-use hazard is detected, the dependent instruction and every instruction behind it are held for one additional cycle while a bubble is inserted, as shown below for LW x22, 12(x0) followed immediately by ADD x23, x22, x1.

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6
LW x22, 12(x0)	IF	ID	EX	MEM	WB	
ADD x23, x22, x1		IF	ID (stall)	ID	EX	MEM
next instruction			IF (stall)	IF	ID	EX

This behaviour was directly observed in simulation: the pipeline monitor recorded Stall_F=1 and Stall_D=1 for exactly one cycle at the load-use hazard point in the test program, with Flush_E=1 in the same cycle to bubble the Execute stage, confirming correct stall/bubble insertion.

4.12 Verification and Testing

Verification was carried out using a directed instruction sequence covering every implemented instruction class, together with an explicit load-use hazard scenario and several back-to-back register-dependency scenarios exercising the forwarding paths. A self-checking Verilog testbench (using check_reg and check_mem tasks) compared the final Register File and data-memory contents in simulation against hand-computed expected values, reporting PASS or FAIL per check.

Test Case	Instruction / Scenario	Expected Result	Actual Result	Status
TC-01	ADDI x1, x0, 10	x1 = 0x0000000A	0x0000000A	PASS
TC-02	ADD x3, x1, x2	x3 = 0x0000001E	0x0000001E	PASS
TC-03	SUB x4, x1, x2 (data-hazard fix)	x4 = 0xFFFFFFFF6	0xFFFFFFFF6	PASS
TC-04	SLT x11, x1, x2	x11 = 0x00000001	0x00000001	PASS
TC-05	LB x17, 0(x0) (sign-extend)	x17 = 0xFFFFFFFF80	0xFFFFFFFF80	PASS
TC-06	LBU x20, 0(x0) (zero-extend)	x20 = 0x00000080	0x00000080	PASS
TC-07	SW x3, 12(x0)	MEM[3] = 0x0000001E	0x0000001E	PASS



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

TC-08	LW x22,12(x0) → ADD x23,x22,x1 (load-use stall)	Stall_F=Stall_D=1 for 1 cycle; x23 correct	Observed as expected	PASS
TC-09	BNE x1,x2,+8 (branch taken, flush)	Flush_D=Flush_E=1; skipped instr. not executed	Observed as expected	PASS
TC-10	JAL x22, +8	x22 = PC+4 = 0x00000084	0x00000084	PASS
TC-11	JALR x24, x23, 0	x24 = PC+4 = 0x00000090	0x00000090	PASS
TC-12	LUI x25, 0x12345	x25 = 0x12345000	0x12345000	PASS
TC-13	AUIPC x26, 0x1 (datapath fix)	x26 = 0x00001098	0x00001098	PASS
TC-14	x0 hard-wired zero	x0 = 0x00000000	0x00000000	PASS

The complete self-checking testbench comprises thirty-one architectural checks (twenty-eight register-file checks and three data-memory checks). All thirty-one checks passed in the final verified version of the design. During development, two functional defects were identified and corrected before this final result was achieved: a same-cycle register-file read/write hazard affecting the SUB instruction (resolved with a write-through bypass in the register file), and a missing datapath path for the AUIPC instruction's PC-relative result (resolved by adding a dedicated PC + immediate result path to the Write-Back multiplexer).

4.13 Results

The verified processor correctly executes R-type arithmetic/logic instructions (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU), I-type arithmetic instructions (ADDI, ANDI, XORI, ORI), all supported load and store widths (LB/LH/LW/LBU/LHU, SB/SH/SW), conditional branches (BEQ, BNE, BLT, BGE), unconditional jumps (JAL, JALR), and the U-type instructions LUI and AUIPC. Data hazards are correctly resolved through EX-stage forwarding from both the Memory and Write-Back stages, with Memory-stage forwarding correctly prioritised. Load-use hazards correctly trigger a one-cycle stall with bubble insertion, and taken branches/jumps correctly flush the two younger in-flight instructions and redirect the Program Counter. All thirty-one self-checking testbench assertions pass.

4.14 GitHub Repository

<https://github.com/sure-trust/SURAJ-P-MADIWAL-g3-25-integrated>
vlsi/tree/421eb538dee583276438e2f0254006cd3f0ea7b3/Final%20capstone%20project



Social / Industry Relevance of the Project

Social Relevance

Open, royalty-free processor architectures such as RISC-V lower the barrier to entry for hardware innovation, particularly for students and organisations in resource-constrained settings who cannot afford proprietary ISA licences. Building practical, hands-on understanding of RISC-V processor design contributes to the broader goal of democratising access to hardware design skills and open computing infrastructure.

Industry Relevance

RISC-V has seen rapid adoption across the semiconductor and embedded-systems industry, including in microcontrollers, IoT devices, accelerators, and increasingly in application-class processors. The skills demonstrated in this project — RTL design in Verilog, pipeline hazard analysis, simulation-based verification using an industry-standard simulator (Synopsys VCS), and systematic hardware debugging — are directly aligned with the core competencies required for RTL design engineer, design verification engineer, and ASIC/FPGA engineer roles in the semiconductor industry.

Practical Applications

- Foundation for embedded microcontroller cores used in IoT and edge devices.
- Teaching and reference implementation for computer architecture and digital design courses.
- Base design for further extension into a synthesisable, FPGA-deployable soft processor core.

Potential Users

- Engineering students and educators studying computer architecture and digital design.
- Hobbyist and academic FPGA developers seeking an open, extensible RISC-V core.
- Early-stage hardware design teams evaluating RISC-V for embedded applications.

Employment / Skill Relevance

Completion of this project developed direct, demonstrable proficiency in Verilog RTL design, pipelined processor architecture, data-hazard forwarding and stall logic, control-hazard flush logic, and simulation-based functional verification using Synopsys VCS — skills that map directly onto entry-level RTL design and design-verification engineering roles in the VLSI and semiconductor industry.



Learning and Reflection

Technical Learning

The project provided practical experience in Verilog HDL coding practices, modular RTL design, and the structural implementation of a five-stage pipelined processor datapath. Significant technical learning was gained in the analysis and resolution of pipeline hazards: implementing EX-stage operand forwarding with correct Memory-stage-over-Write-Back-stage priority, detecting and stalling for load-use hazards, and flushing the pipeline correctly on taken branches and jumps. The project also provided hands-on experience with simulation-based debugging using Synopsys VCS, including the use of cycle-by-cycle pipeline-monitor traces to localise functional defects to a specific signal and pipeline stage, and the design and development of a self-checking testbench to automate verification and prevent regression.

Project Management Learning

The project required systematic planning of the module-by-module implementation sequence (Fetch, Decode, Execute, Memory, Write-Back, and the supporting Hazard Unit), followed by integration and instruction-class-by-instruction-class verification. Debugging the SUB-instruction and AUIPC defects required disciplined, evidence-based root-cause analysis — forming and testing specific hypotheses against simulation waveforms rather than making speculative changes — which reinforced the importance of methodical problem decomposition.

Overall Experience

The most significant learning outcome of this project was the practical experience of debugging a subtle, timing-dependent hardware defect: the SUB-instruction failure initially appeared to be specific to the subtraction operation, but systematic signal-level tracing revealed that the true cause was a same-cycle register-file read/write collision affecting any instruction in an equivalent pipeline position — SUB was simply the instruction that happened to occupy that position in the test sequence. This experience reinforced the importance of tracing a defect to its true root cause across pipeline stages, rather than accepting the first plausible explanation, and directly demonstrated why self-checking, automated verification is preferable to manual inspection of simulation logs for catching regressions such as the subsequently discovered AUIPC datapath omission.



Conclusion and Future Scope

Conclusion

This project set out to design, implement, and functionally verify a five-stage pipelined RV32I RISC-V processor core in Verilog HDL, with correct handling of data and control hazards. All targeted instruction classes — R-type, I-type arithmetic, load, store, branch, jump, and U-type instructions — were implemented across a modular datapath comprising sixteen independent Verilog modules integrated through a top-level design. Verification using Synopsys VCS identified two functional defects: a same-cycle register-file read/write hazard affecting the SUB instruction, and a missing datapath for the AUIPC instruction's PC-relative result. Both defects were root-caused through systematic pipeline-trace analysis and corrected — the first via a write-through bypass in the register file, and the second via a new result-source path threading the PC + immediate sum through the pipeline into the Write-Back multiplexer. A thirty-one-check self-checking testbench was subsequently developed and confirmed full architectural correctness across all implemented instruction classes and hazard scenarios.

Future Scope

- Carry the design through logic synthesis and FPGA deployment to validate timing closure and real hardware operation.
- Extend the instruction set to cover remaining RV32I instructions (FENCE, ECALL/EBREAK) and optional standard extensions such as M (multiplication/division).
- Replace the behavioural instruction/data memory models with a synthesisable memory subsystem and a standard memory-mapped bus interface.
- Add exception and interrupt handling to support a minimal operating-system-capable core.
- Introduce branch prediction to reduce the control-hazard penalty currently incurred on every taken branch/jump.
- Expand the self-checking testbench into a structured regression suite with directed and randomised test generation.



5-Stage Pipelined RISC-V Processor Datapath :-

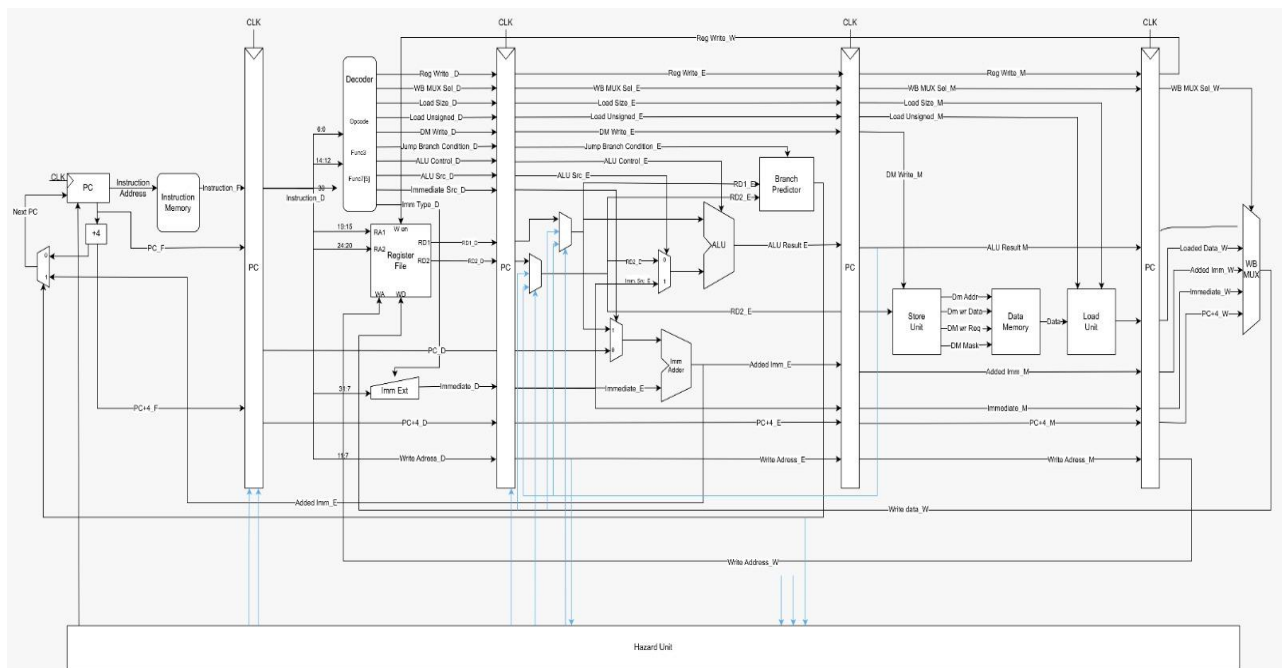


Figure: 5-Stage Pipelined RISC-V RV32I Processor Datapath



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

References

- [1] RISC-V International, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” official ISA specification.
- [2] S. L. Harris and D. M. Harris, Digital Design and Computer Architecture: RISC-V Edition, Morgan Kaufmann.
- [3] Synopsys, Inc., “VCS User Guide,” Synopsys VCS Documentation, version X-2025.06-SP1.
- [4] IEEE Std 1364, “IEEE Standard for Verilog Hardware Description Language,” Institute of Electrical and Electronics Engineers.
- [5] SURE Trust – Innovation & Entrepreneurship Hub for Educated Rural Youth (IERY), internship programme and mentorship material.